



UFC

UNIVERSIDADE FEDERAL DO CEARÁ
CAMPUS JARDINS DE ANITA - ITAPAJÉ

Reconhecimento de Padrões: Um Sistema Que Classifica Palavras Através da Transformada Rápida de Fourier

Carlos Eduardo Teles Alencar
Giovanna Dias Castro de Oliveira
Jônatas Fernandes Silva
Leandro Nascimento Adegas
Victor Matheus Araújo Oliveira

Disciplina: Aprendizado de Máquina e Reconhecimento de Padrões

Itapajé, Julho, 2025



Conteúdo

1	Introdução	3
2	Background e Trabalhos Relacionados	4
2.1	Transformada Rápida de Fourier	4
2.2	Codificação Preditiva Linear	5
2.3	<i>Perceptron</i> Multicamadas	5
3	Modelo de Solução	5
3.1	Construção do Sistema	6
3.2	Modelo de Classificação	6
4	Metodologia	6
4.1	O Ambiente de Desenvolvimento	7
4.2	Passo a Passo da Escrita do Sistema	7
4.2.1	Telas	7
4.2.2	Servidor	7
4.2.3	Módulo de Aprendizado de Máquina	8
4.3	Etapas da Análise de Dados	8
5	Resultados	8
5.1	Sinais do Áudio de Cada Palavra	8
5.2	Análise dos Componentes Principais de cada Palavra	9
5.3	Métricas de Avaliação do Modelo	10
6	Conclusão	12
6.1	Contribuição	12
6.2	Limitações e Trabalhos Futuros	13
7	Agradecimentos	13
8	Referências	14



Resumo

Este trabalho tem por objetivo o desenvolvimento um sistema de identificação de palavras por meio da transformada rápida de fourier (fast fourier transform, FFT). As palavras escolhidas foram: *brincar, comer, corrida, entrar, partida, procurar, sair e testar*, estas palavras são ditas por um usuário, através de uma página web responsiva. A partir deste áudio enviado ao sistema, inicia-se o processo de análise do mesmo para que possa haver a extração da informação contida nele. A contribuição deste trabalho está no desenvolvimento de um sistema que consegue identificar a palavra dita por um usuário em 62,5% das vezes. Em nossos resultados, pudemos notar que, o ambiente onde é realizado a gravação, isto é, o envio do audio tem influência na precisão da análise do sistema e que por mais que o computador consiga aprender e reconhecer padrões, nunca irá ser realmente inteligente, pois isto é algo inerente ao ser humano.

1 Introdução

A análise de som é um desafio na área de aprendizado de máquina (machine learning, ML). Porém, é um campo importantíssimo e que tem avançado bastante nos últimos tempos. Som nada mais é do que um conjunto de frequências envolta por em um espaço de tempo, portanto, podemos dizer que a análise de som envolve uma visão espacial e temporal. Partindo dessa ideia, sons são séries temporais, ou seja, podem ser avaliadas através de modelos de classificação já conhecidos, como *long short-term memory* (LSTM) [1], ou com *recurrent neural network* (RNN), até mesmo utilizando *multi layer perceptron* (MLP) como em [2].

Sabendo disto, é importante citar que, o processamento e o reconhecimento são imprescindíveis [2]. Temos frameworks como o *Nvidia NeMo*, que tem alta capacidade para o processamento de áudio, mas que, contudo, não iremos utilizar neste trabalho para fins de pesquisa. Neste estudo não será feito o uso de bibliotecas externas para o processamento e análise do som. A ferramenta utilizada aqui será FFT.

O objetivo principal deste trabalho é construir um sistema capaz de identificar comandos, palavras ditas por um usuário. Denota-se que, os áudios utilizados para o treinamento do modelo escolhido para o estudo foram gravados por mais de 10 pessoas diferentes, em ambientes diferentes, isto é, os áudios que serão analisados aqui tem frequências de voz diferentes. Por esta razão optou-se por não realizar a FFT em todo o sinal, mas em cada quadro dentro do sinal, ao final recuperando-o por completo.

Este estudo foi provocado pelo professor Dr. Júlio César Santos do Anjos, em uma de suas aulas. Como parte da segunda avaliação ("P2") da disciplina de aprendizado de máquina e reconhecimento de padrões. Motivado pelo aprendizado da análise de séries temporais e por um estudo futuro que realizar-se-á, este trabalho é uma forma de nos familiarizarmos com análise e reconhecimento de padrões em sons. Além do incentivo ao conhecimento de como esse processo ocorre por baixo dos panos.

O trabalho base para este estudo é de autoria do Dr. Igor R. Sousa, intitulado como "Revisitando o Uso da Transformada de Fourier no Reconhecimento de Voz para Robótica Móvel" [2]. Este trabalho propõe um sistema embarcado em um pequeno robô. Seu objetivo é a identificação de cinco comandos, a saber, **avançar, direita, esquerda, parar e recuar**. Neste estudo também não utilizou-se bibliotecas externas, o que torna possível o uso offline.

Durante o desenvolvimento do sistema notou-se que, a qualidade das amostras influencia diretamente no treinamento do modelo de classificação, isto é, as etapas anteriores ao treinamento do modelo são fundamentais para o sucesso da análise. Caso não haja uma boa preparação dos dados, os resultados sempre serão negativos. As etapas da análise dos dados, citadas pelo professor Dr. Júlio César [3], devem ser seguidas a risca. Além disso, notamos que por mais que o modelo consiga aprender bem, ele não é inteligente, pois, como um atributo inato concedido a humanidade, nem mesmo sob base empírica de testes um modelo estatístico é capaz de replicar a inteligência real.



A organização deste trabalho segue a seguinte ordem. A seção 2 contém os fundamentos que foram necessários para o desenvolvimento do sistema, e conceitos que são considerados importantes para o bom entendimento deste trabalho. A seção 3 explica o modelo de solução proposto para a resolução do problema de análise de voz através da FFT. A seção 4 explica o ambiente e os métodos utilizados para as demonstrações empíricas deste trabalho. A seção 5 traz os resultados obtidos após a aplicação do modelo de solução. Por fim, a seção 6 traz a conclusão, contribuição e limitações deste trabalho, além de possíveis trabalhos futuros.

2 Background e Trabalhos Relacionados

Esta seção apresenta os principais conceitos relacionados a transformada rápida de fourier e outros trabalhos que tem sido desenvolvidos para demonstrar as diversas formas de uso da FFT. Apontando a importância da análise de sons para a computação moderna.

2.1 Transformada Rápida de Fourier

A transformada rápida de fourier é uma ferramenta matemática que realiza a transição entre as variáveis de tempo e frequência [4], dada por

$$F(\omega) \equiv \mathcal{F}\{f(t)\} = \int_{-\infty}^{\infty} f(t) e^{-i\omega t} dt \quad (1)$$

diferentemente do trabalho de [4], para este estudo o foco se deu na amplitude, trabalhando na dimensão tempo x espaço. Na transformada rápida de fourier o domínio é o das frequências que formam o sinal, como na Figura 1. Destaca-se que, ao aplicar a FFT, a amplitude é dada em módulo, isto é, apenas valores positivos são utilizados. Adicionalmente, é fundamental citar que usa-se apenas os picos das frequências, assim como em [2].

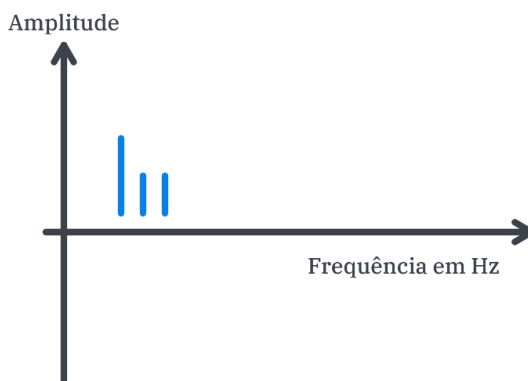


Figura 1: Aplicação Simulada da FFT.

Sabendo disto, como foi dito na seção 1, o trabalho base para a realização deste estudo é intitulado “Revisitando o Uso da Transformada de Fourier no Reconhecimento de Voz para Robótica Móvel” [2]. Nesse artigo, o autor utiliza a FFT de uma forma não usual, aplicando-a sobre todo o sinal. Para seu experimento, ele utilizou dois classificadores, um linear, discriminante linear dos mínimos quadrados (LMQ) e *multilayer perceptron* (MLP). Como resultado obteve que, *FFT+LMQ* teve mediana de 96,94%, e para *FFT+MLP* obteve mediana igual a 100%. Nosso diferencial em relação a este artigo se dá pelo seguinte, demonstrado na Tabela 1. Reforçando que, uso da FFT em quadros se dá porque a voz não é um dado estacionário, logo usá-la em todo o sinal prejudicaria o modelo treinado e o reconhecimento dos padrões nas frequências sonoras de cada áudio fornecido no dataset.



Tabela 1: Diferenciais

Trabalho do Dr. Igor	Nosso Trabalho
Áudios gravados por uma única pessoa	Áudios gravados por mais de 10 pessoas
Embarcado em um robô	Sistema web responsivo
Usa FFT em todo o sinal	Usa FFT em segmentos curtos (quadros)

2.2 Codificação Preditiva Linear

A técnica de codificação preditiva linear (*linear predictive coding*, LPC), é um método utilizado para o processamento de sinal de áudio e processamento de fala [5]. Sua equação é dada por

$$s(n) = \sum_{k=1}^p a_k \cdot s(n-k) + e(n) \quad (2)$$

onde o LPC analisa o sinal de fala estimando os formantes, após isso ele remove os efeitos do sinal de fala e estima a amplitude e frequência do que sobra após a remoção dos efeitos. Por causa da variação do sinal no tempo, esse processo é realizado em quadros do sinal.

2.3 Perceptron Multicamadas

O classificador *perceptron* multicamadas (*multilayer perceptron*, MLP), é um modelo de rede neural artificial (RNA) formada por multiplas camadas de *perceptrons*, como o nome já deixa explícito. Apenas a título de informação, um perceptron é uma função de limiar aplicada ao componente de um vetor [6]. Uma rede MLP é formada por uma camada de entrada, camadas ocultas e uma camada de saída, como no exemplo abaixo.:

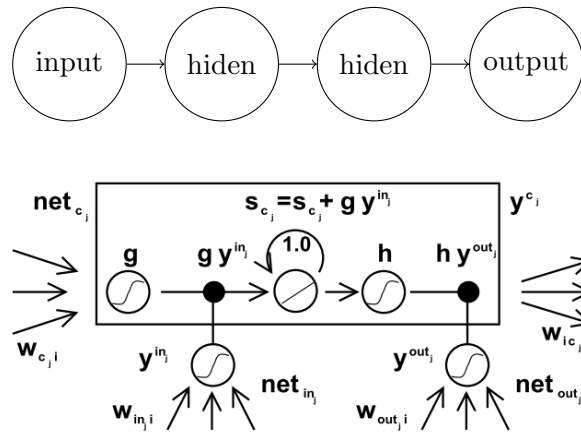


Figura 2: Arquitetura da Célula [7]

3 Modelo de Solução

Nessa seção será demonstrado e explicado o nosso modelo de solução para o problema proposto na introdução do estudo. Primeiramente será visto a modelagem do sistema, em seguida será explicado como cada componente se comporta, suas responsabilidades e como se dá o uso da FFT e do classificador escolhido para este trabalho.



3.1 Construção do Sistema

A construção do sistema foi feita em partes. O *frontend*, isto é a parte visual, foi desenvolvido utilizando o *framework* *Angular* na sua versão 20 (vinte). Essa parte é subdividida em componentes e serviços, dentro dessas subdivisões é utilizado *Hypertext Markup Language* (HTML), *Cascading Style Sheets* (CSS) e *TypeScript*. Para fazer a conexão com o *backend*, desenvolvido em python, por meio do *framework* *FastAPI*, foi utilizado apenas módulos *Hypertext Transfer Protocol* (HTTP) do lado do cliente e *Cross-Origin Resource Sharing* (CORS) do lado do servidor. Na Figura 3 é exibida a arquitetura simplificada do sistema, onde o cliente se comunica com o servidor através da interface de programação da aplicação (*application programming interface*, API) e por meio da API o modelo de classificação desenvolvido, doravante, modelo.

Modelo de Design do Sistema

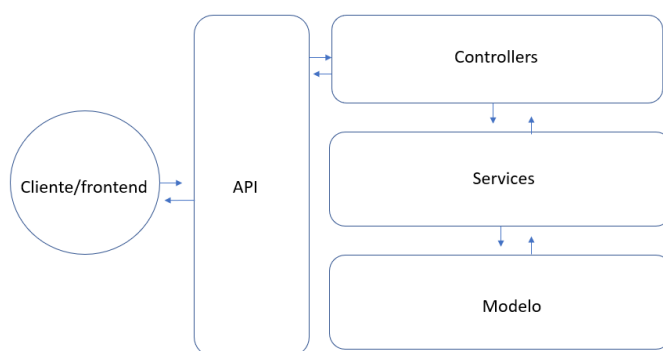


Figura 3: Design da Arquitetura Simplificada do Sistema.

3.2 Modelo de Classificação

Para o classificador foi aderido o esquema ilustrado na Figura 4. O fluxo no módulo de classificação funciona da seguinte forma, ao receber a entrada de dados, isto é, o áudio enviado pelo usuário, este arquivo é recebido pelo servidor, já descrito acima, na Figura 3; ao ser recebido ele é enviado ao módulo de classificação, onde ele será pré-processado, passando pelo LPC e FFT para a extração das features, é normalizado utilizando a técnica *boxcox* e é feita a análise dos componentes principais (PCA), por fim ele vai para o modelo já treinado para que o mesmo venha a classificar o áudio em *brincar*, *comer*, *corrida*, *entrar*, *partida*, *procurar*, *sair* ou *testar*. Após este processo o modelo devolve a resposta em *JavaScript Object Notation* (JSON) para o servidor que por sua vez devolve para o cliente.

4 Metodologia

Nesta seção serão apresentadas as condições e a maneira que ocorreu o processo de desenvolvimento do modelo de solução. As etapas para este processo metodológico se dividem em, descrição do ambiente de desenvolvimento e o passo a passo da escrita do sistema, que por sua vez é dividido em três partes, são elas: telas, servidor e modelo de classificação, ou módulo de aprendizado de máquina. Para o módulo de aprendizado de máquina foram realizadas todas as etapas da análise de dados mencionadas nos slides de aula do professor Dr. Júlio César [3].



Sistema para Classificação de Palavras com FFT

Modelo de Design do Classificador

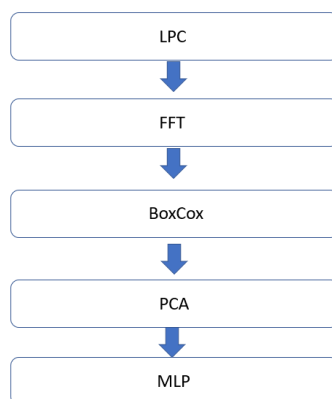


Figura 4: Design da Arquitetura Simplificada do Classificador.

4.1 O Ambiente de Desenvolvimento

Para o desenvolvimento deste sistema foram utilizados três ambientes de programação. Para a escrita do código das telas foi usado o ambiente de desenvolvimento (IDE) *WebStorm*, já o servidor foi feito na IDE *pycharm* e o modelo de aprendizado de máquina foi escrito e testado isoladamente no *google colab*. Sendo que, os três módulos foram construídos individualmente e ao final integrados. Para armazenar o código integrado, utilizamos o *github*, o repositório para acessar os códigos é <https://github.com/jonatasfernandessilva7/P2-analise-e-reconhecimento-de-padrones.git>.

4.2 Passo a Passo da Escrita do Sistema

Nesta subseção será apresentado o passo a passo para o desenvolvimento de todo o sistema utilizado para a realização deste trabalho.

4.2.1 Telas

Para o desenvolvimento das telas do sistema utilizamos o *framework Angular*, na sua versão 20 (*latest*). A escolha desta ferramenta deu-se por motivo de robustez e sintaxe, apesar de ser um projeto relativamente pequeno. Estas telas são divididas em componentes e serviços. Os componentes são responsáveis pela renderização das telas, enquanto os serviços são responsáveis por realizar a conexão com o servidor através dos métodos HTTP. Como o nosso sistema possui somente duas telas, sendo que uma delas é apenas uma tela de "bem-vindo"; consome duas rotas do servidor, rota de iniciar gravação e parar gravação, todo o restante do processamento é realizado no servidor.

4.2.2 Servidor

O servidor foi desenvolvido utilizando a linguagem *python*, por meio do *framework FastAPI*. A escolha desta ferramenta deu-se pela maior facilidade de integração com o módulo de aprendizado de máquina, que seria escrito em *python*. O servidor é dividido no arquivo central, chamado "*server.py*" um único arquivo de controle da API e os serviços, que são no caso, os módulos de aprendizado de máquina divididos em arquivos únicos, além da própria API. Como não foi trabalhado o uso de banco de dados (BD) dado o nível de complexidade do estudo, tudo o que precisava ser salvo foi salvo em memória interna.



4.2.3 Módulo de Aprendizado de Máquina

Este módulo foi escrito em *python*. Como dito a cima, essa etapa do desenvolvimento após a integração foi dividida em serviços. Esses serviços são: análise dos componentes principais, *boxcox*, FFT, LPC MLP e um arquivo de treino, onde o modelo é treinado e salvo por meio da biblioteca *joblib*, para que possa ser usado pelo controlador quando os áudios são recepcionados do *frontend*. Dentro do FFT e do LPC não foram utilizadas bibliotecas externa, sendo isto um dos requisitos necessários para a validação do projeto.

4.3 Etapas da Análise de Dados

Para que o módulo de aprendizado de máquina se comportasse como esperado e retornasse bons resultados é necessário que todas as etapas de análise de dados mostrados em [3] sejam seguidas. Essas etapas são ilustradas pela Figura 5. Os dados foram selecionados do dataset feito pelas vozes de todos os alunos da disciplina. Após a seleção dos áudios utilizados realizamos o pré-processamento, logo após isto realizamos a extração das *features* via LPC e FFT, normalizamos eles por meio do *boxcox*, realizamos a análise dos componentes principais via PCA e treinamos o modelo de classificação.

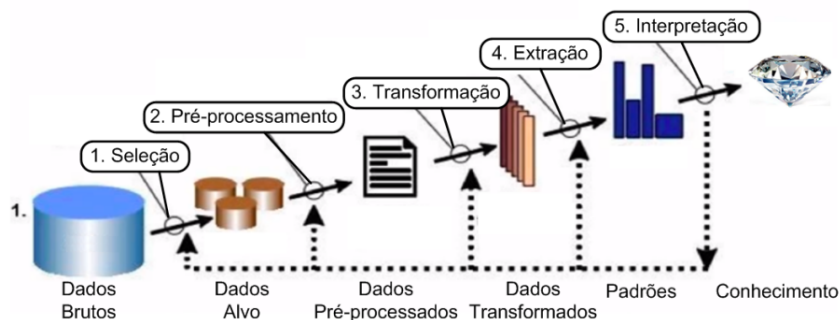


Figura 5: Etapas da Análise de Dados.

5 Resultados

Nesta seção serão apresentados todos os resultados obtidos após o treinamento do modelo. Veremos o resultado após o sistema desenvolvido, métricas imagens que ilustram o desempenho do modelo de classificação.

5.1 Sinais do Áudio de Cada Palavra

A Figura 6 mostra os sinais de áudio (forma de onda) utilizados para treinamento, com um exemplo por classe. Cada classe representa uma palavra distinta dita por diferentes usuários. Os sinais exibem variações significativas de amplitude e duração, que justificam a aplicação da FFT em janelas ao longo do tempo.

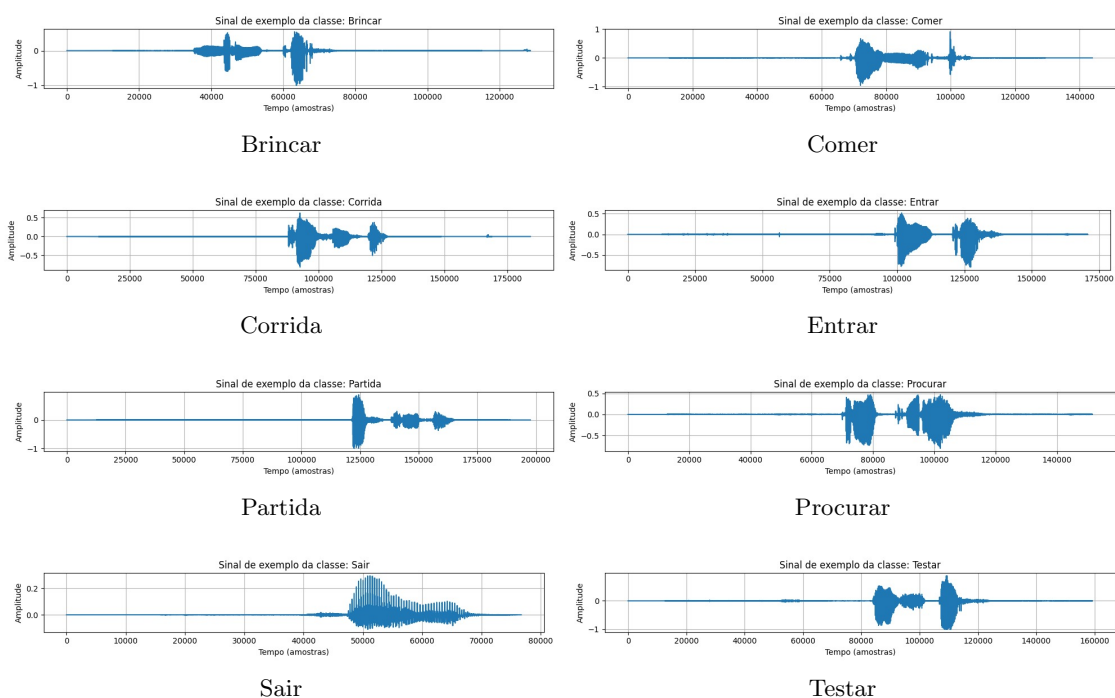


Figura 6: Sinais de exemplo por classe de comando de voz.

5.2 Análise dos Componentes Principais de cada Palavra

Além da visualização dos sinais brutos no domínio do tempo, realizamos também a transformação espectral dos áudios das diferentes classes. Essa transformação foi feita por meio da aplicação da Transformada Rápida de Fourier (FFT), seguida da análise de componentes principais (PCA), com o intuito de reduzir a dimensionalidade e destacar os padrões mais relevantes. A Figura 7 representa o espectrograma reduzido via PCA para cada uma das classes.

Nesses espectrogramas, o eixo horizontal representa o tempo (em quadros), o eixo vertical representa os principais componentes extraídos, e as cores indicam a intensidade das componentes de frequência ao longo do tempo. Essa representação espectral, reduzida por PCA, foi fundamental para alimentar o classificador com dados mais expressivos e menos redundantes. A análise visual reforça a existência de padrões únicos por classe, o que contribuiu para a acurácia do modelo MLP utilizado.

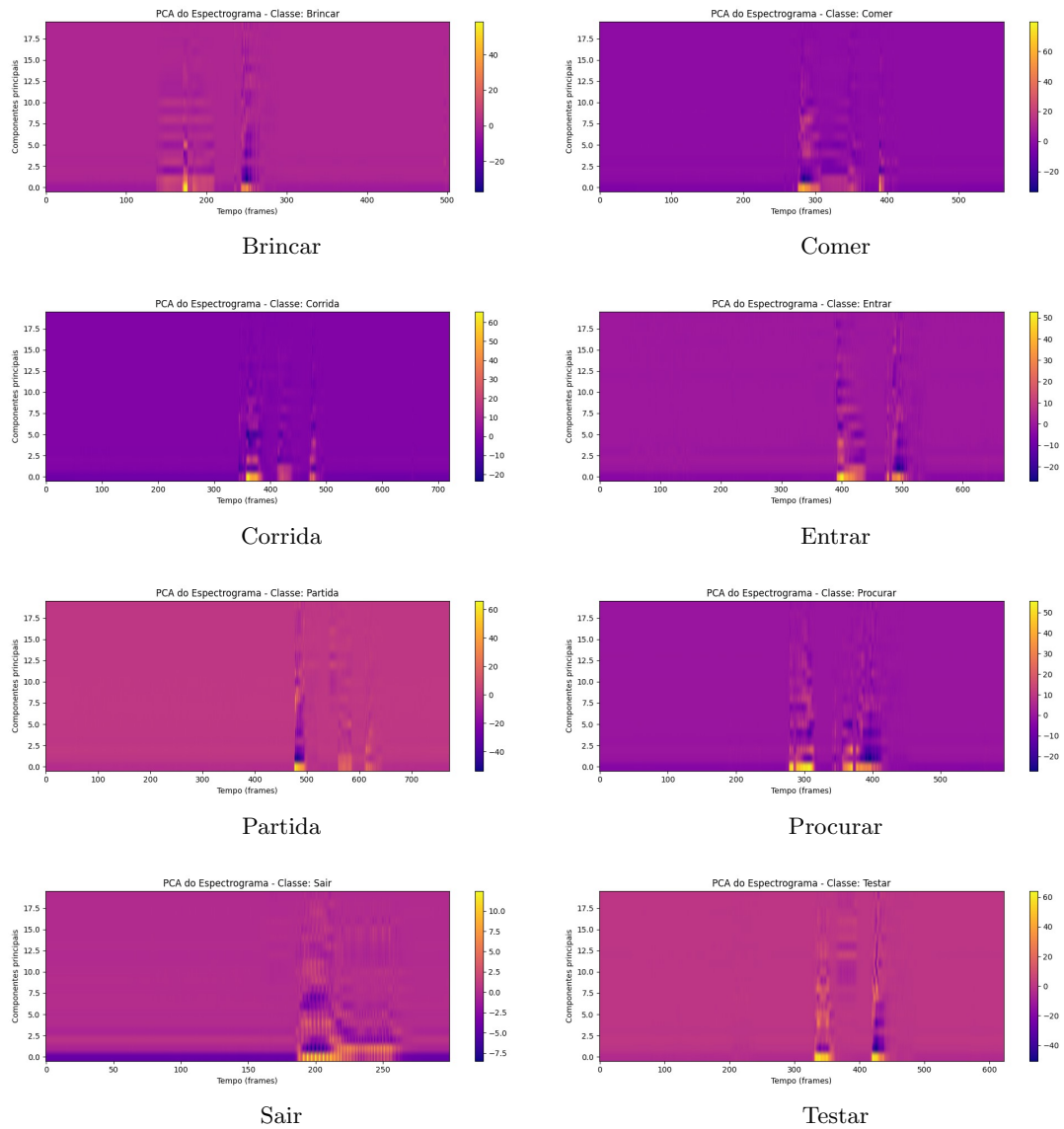


Figura 7: PCA do Espectrograma de Cada Palavra.

5.3 Métricas de Avaliação do Modelo

Adicionalmente, realizamos uma avaliação quantitativa do desempenho do modelo MLP otimizado. A Figura 8 apresenta a curva de acurácia por época, evidenciando a evolução positiva da acurácia tanto para o conjunto de treino quanto de validação. A convergência é alcançada após 40 épocas, com acurácia estabilizada em torno de 96%, o que indica um bom balanceamento e ausência de *overfitting* significativo.



Sistema para Classificação de Palavras com FFT

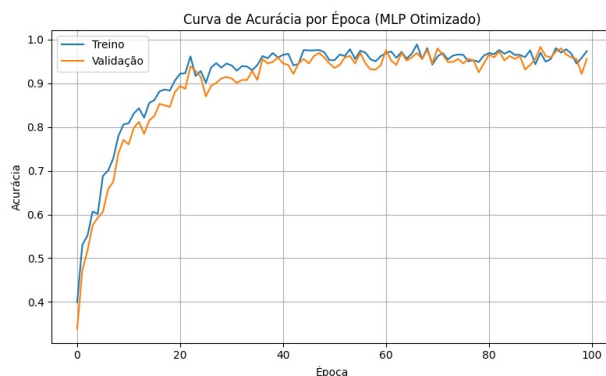


Figura 8: Curva de Acurácia por Época.

Apesar dessas inconsistências pontuais, o modelo se mostrou eficaz, com *f1-score* médio de 0.96, conforme apresentado na Tabela 2. O detalhamento dos indicadores de desempenho por classe mostra que:

1. A palavra Procurar teve o melhor desempenho geral, com precision, recall e *f1-score* próximos de 1.00;
2. A classe Brincar, embora com precisão perfeita (1.00), teve *recall* mais baixo (0.86), indicando que alguns exemplos foram confundidos com outras classes;
3. As palavras Entrar e Testar foram corretamente classificadas em 100% dos casos (*recall* 1.00).

a Figura 9 confirma que os erros cometidos foram concentrados em três das oito classes, reforçando a robustez geral do sistema, mesmo com variações nos timbres e ambientes de gravação.

Classe/Acerto	S	N
Brincar	S	N
Comer	S	N
Corrida	S	N
Entrar	N	Entrar
Partida	N	Entrar
Procurar	S	N
Sair	S	N
Testar	N	Sair

Figura 9: Acerto das Palavras.

Tabela 2: Métricas por classe: precisão, revocação e F1-score

Classe	Precisão	Revocação	F1-score	Suporte
Brincar	1.00	0.86	0.93	36
Comer	1.00	0.94	0.97	36
Corrida	0.95	0.97	0.96	37
Entrar	0.95	1.00	0.97	37
Partida	0.86	0.97	0.91	37
Procurar	0.97	1.00	0.99	36
Sair	1.00	0.89	0.94	37
Testar	0.95	1.00	0.97	36

A Figura 10 mostra a matriz de confusão do modelo obtida com a base de testes. A diagonal principal representa as classificações corretas; valores fora da diagonal evidenciam confusões entre palavras, especialmente entre classes foneticamente semelhantes.

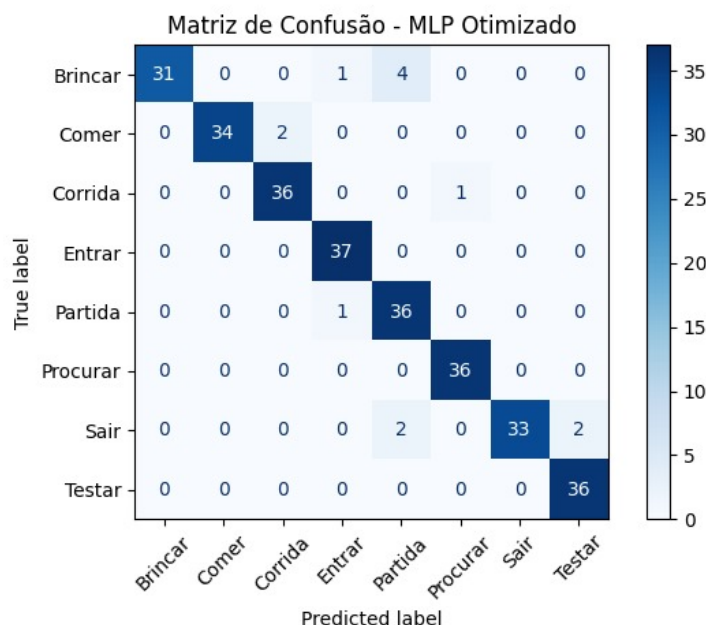


Figura 10: Matriz de confusão do modelo de classificação.

A matriz de confusão mostra que não há um *overfitting* claro, porém, há confusão em testes reais com dados não vistos no modelo. Isto pode ser atribuído à baixa variabilidade de amostras por classe. O modelo mesmo após muitos ajustes e mesmo após testar vários modelos ele continua cometendo erros devido à baixa variabilidade, como mostrado na Figura 7, isso ocorre pois as faixas de ativação de voz são muito semelhantes mudando apenas o comprimento. Por esta razão o modelo foi testado com *augmentation*, foram usados: *foram usados*, *add_noise*, *pitch_shift* e *time_stretch*.

Para ver o sistema funcionando com áudios enviados por um usuário real, gravamos um vídeo que está disponível em <https://drive.google.com/file/d/1QM5HsaAkho1cEC16Vhnu62YpubRNKjfq/view?usp=sharing>. Neste vídeo demonstramos na prática tudo o que já foi mencionado, e as limitações que o modelo encontra em relação à análise de múltiplas vozes. Dado a sua limitação de aprendizado, e sua incapacidade de ser inteligente de fato.

6 Conclusão

Os resultados obtidos comprovam a robustez do sistema, alcançando uma acurácia média de 96% e *f1-score* igualmente elevado. As análises visuais dos sinais e espectros, bem como as métricas de desempenho por classe, evidenciam que o modelo foi capaz de identificar corretamente a maioria dos comandos, mesmo diante da variabilidade de timbre, entonação e ruídos dos ambientes de gravação. O bom desempenho nas classes mais desafiadoras indica que o sistema foi bem treinado e validado. Porém, mesmo que o modelo esteja bem treinado, quando vai para um ambiente real, ele sofre devido à variação de frequências vocais.

6.1 Contribuição

Este trabalho contribui de forma significativa para o entendimento e a aplicação prática de técnicas de reconhecimento de padrões em sinais de voz. A principal contribuição reside na construção de um sistema funcional de classificação de palavras, que une uma cadeia de pré-processamento de áudio robusta (FFT, LPC, Box-Cox, PCA) a uma rede MLP, integrado a uma interface web responsiva. Diferentemente de abordagens anteriores, que trabalhavam com sinais de uma única



fonte ou usavam FFT no sinal completo, o presente sistema utilizou gravações de múltiplos usuários e aplicou a FFT por janelas (quadros), junto com LPC, dado a natureza não estacionária da voz.

6.2 Limitações e Trabalhos Futuros

Apesar dos bons resultados obtidos, o sistema apresenta algumas limitações. A principal delas é a sensibilidade à qualidade e à variação dos áudios de entrada. Ambientes ruidosos, microfones com qualidade inferior ou usuários com pronúncia distinta da média podem prejudicar a acurácia do modelo. Além disso, algumas palavras foneticamente semelhantes ainda geram confusões, como evidenciado na matriz de confusão entre as classes “Brincar” e “Partida” ou “Sair” e “Entrar”.

Isso sugere que o modelo poderia se beneficiar de uma arquitetura mais sensível a sutilezas fonéticas, como redes neurais recorrentes (RNN) ou LSTM, que tratam explicitamente a sequência temporal dos dados. Como trabalhos futuros, sugerem-se: Inclusão de novos algoritmos de classificação, como LSTM ou CNNs, para comparação de desempenho e aumento do dataset com mais amostras por classe e gravações em diferentes contextos.

7 Agradecimentos

Agradecemos primeiramente a Deus pela inspiração e guia, ao professor Júlio César Santos dos Anjos pela orientação e dedicação, e aos valorosos colegas pela parceria e contribuições.



8 Referências

- [1] J. C. S. dos Anjos, “Reconhecimento de padrões – análise de séries temporais.” Aula ou apresentação acadêmica, 2025.
- [2] I. R. Sousa, J. C. Figueiredo, and G. A. Barreto, “Revisitando o uso da transformada de fourier no reconhecimento de voz para robótica móvel,” *Defesa de Doutorado, PPGETI, UFC*, 2025.
- [3] J. C. S. dos Anjos, “Introdução ao aprendizado de máquina e reconhecimento de padrões.” Aula ou apresentação acadêmica, 2025.
- [4] L. A. Golçalves, “Um estudo sobre a transformada rápida de fourier e seu uso em processamento de imagens,” *Universidade Federal do Rio Grande do Sul*, 2004. Dissertação de mestrado.
- [5] Wikipedia contributors, “Linear predictive coding — wikipedia, the free encyclopedia.” https://en.wikipedia.org/wiki/Linear_predictive_coding, 2024. Accessed: 2025-07-25.
- [6] J. C. S. dos Anjos, “Algoritmos e técnicas de classificação.” Aula ou apresentação acadêmica, 2025.
- [7] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, 9(8): 1735-1780, 1997.